DevOps, CI/CD & Monitoring

Module 5 - Day 21 to Day 25

Project Documentation

Prepared by: **<Your Name>** | Intern, <Company Name>
GitHub: github.com/<your-username>/<your-repo>

1. Overview

This project implements an automated CI/CD pipeline for a containerized web application using GitHub Actions. Every code push triggers automated build, test, Docker image creation and deployment, followed by basic monitoring through logs and health checks. The goal is faster, safer and repeatable software delivery.

GitHub > Build > Test > Docker > Deploy > Monitor

- Understand **DevOps culture** and Git/GitHub workflows
- Build **automated CI/CD pipelines** (testing, build and deployment automation)
- Manage **environments** and secrets securely
- Integrate **monitoring** with basic logs and observability

2. DevOps Culture

DevOps unites development and operations to deliver software faster and more reliably through collaboration, automation and continuous feedback.

Principle	Description
Collaboration	Dev and Ops share ownership of the full lifecycle
Automation	Build, test and deploy steps run without manual effort
Continuous feedback	Fast test results and monitoring guide improvements
Infrastructure as Code	Environments defined and versioned in code
Continuous improvement	Small, frequent releases reduce risk

3. Git & GitHub Workflow

- **main** branch always holds production-ready code.
- Work is done in **feature branches** (e.g., feature/add-login).

- Changes are merged through **Pull Requests** with automated checks and review.
- Merging to main triggers the **deployment pipeline**.

```
git checkout -b feature/add-health-check
git add .
git commit -m "feat: add health check endpoint"
git push origin feature/add-health-check
# Open a Pull Request -> CI runs -> review -> merge to main
```

4. CI/CD Concepts

Term	Meaning
Continuous Integration (CI)	Code is merged often; each change is automatically built and tested
Continuous Delivery	Every passing build is ready to be released
Continuous Deployment (CD)	Passing builds are deployed to production automatically

5. Pipeline Stages

Stage	What happens
1. GitHub	Push or Pull Request triggers the workflow
2. Build	Install dependencies and compile / package the app
3. Test	Run automated unit tests; failure stops the pipeline
4. Docker	Build the Docker image and push it to the registry
5. Deploy	Pull the new image on the server and restart the service
6. Monitor	Check logs and health endpoint to verify the release

6. GitHub Actions Workflow

File: `.github/workflows/ci-cd.yml`

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run build --if-present
      - run: npm test

  docker:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ${{ secrets.DOCKER_USERNAME }}/myapp:latest

  deploy:
    needs: docker
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          script: |
            docker pull ${{ secrets.DOCKER_USERNAME }}/myapp:latest
            docker compose up -d
```

7. Environment Management

- Secrets (Docker credentials, SSH keys) are stored in **GitHub Secrets**, never in code.
- Configuration is injected through **environment variables** (.env files stay out of Git).
- Separate **development, staging and production** environments keep releases safe.
- GitHub **Environments** can require manual approval before production deploys.

8. Monitoring & Observability

Monitoring confirms that a deployment is healthy and helps detect issues early.

Pillar	Purpose	Basic Tooling
Logs	Record what the application is doing	docker logs, GitHub Actions logs
Metrics	Measure performance (CPU, memory, requests)	docker stats
Health checks	Verify the service is running	/health endpoint, HEALTHCHECK

```
# View live application logs
docker logs -f <container>

# Check resource usage
docker stats

# Verify service health
curl http://localhost:3000/health
```

Health check configured in Docker Compose:

```
healthcheck:
  test: ["CMD", "wget", "-qO-", "http://localhost:3000/health"]
  interval: 30s
  timeout: 5s
  retries: 3
```

9. Key Learnings

- Automation removes manual errors and speeds up releases.
- Automated tests act as a safety gate before every deployment.
- Secrets must be managed securely and never committed to the repository.
- Monitoring and logs are essential to confirm and troubleshoot releases.
- Small, frequent changes with CI/CD lower risk and improve quality.